Ders 14 Genişletilmiş Notu: Kesişim Kümesi, Akış Diyagramından C'ye ve Matrisler

Ders 14 Genişletilmiş Notu: Kesişim Kümesi, Akış Diyagramından C'ye ve Matrisler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Örnek Soru: İki Kümenin Kesişimi
- 2. İç İçe İki For ile Akış Diyagramı
- 3. İç Döngüyü While'a Çevirmek
- 4. Hangi Küme Dışta, Hangisi İçte Olmalı?
- 5. Akış Diyagramından C Koduna
- 6. Çok Boyutlu Diziler ve Matrisler
- 7. Örnek Soru: Matrisin Kolon Toplamları
- 8. Örnek Soru: Matris Simetrik mi?
- 9. Scanf ve Printf Üzerine Son Sorular

1. Örnek Soru: İki Kümenin Kesişimi

Dersin ilk örneği şudur: verilen iki kümenin kesişim kümesini bulan algoritmanın akış diyagramını çizip C dilinde kodunu yazınız. Hoca soruyla birlikte iki küme örneği verdi: $A = \{1, 3, 5, 2, 4\}$ ve $B = \{6, 1, 7, 2\}$.

1.1 Örnek Değil, Genel Çözüm

Hoca soruya geçmeden önce çözüm yazarken uyulacak iki ilkeyi anlattı. Birincisi, soruda verilen kümeler yalnızca bir örnektir. Burada A'nın 5, B'nin 4 elemanı var diye eleman sayılarını programa 5 ve 4 olarak yazmak yanlıştır. İki küme veriliyorsa uzunlukları birbirinden farklı olabilir ve bu uzunluklar da girdinin parçasıdır.

> **Hocanın vurgusu:** Her zaman genel çözüm beklenir. Örnekteki eleman sayısına ya da örnekteki elemanlara göre çözüm üretmeyin.

İkinci ilke analizle ilgilidir. Analiz istendiğinde, soruda bir örnek varsa analiz o örnek üzerinde yapılır. Örnek yoksa kendi örneğinizi göstererek yaparsınız. Analiz istenmese bile çözümünüzün doğruluğunu görmek için yapabilirsiniz. Fazladan analiz yapmak hiçbir sorun yaratmaz.

1.2 Önce Kağıt Üzerinde Düşünmek

Öğrenciler önce okuma kısmını söyledi: birinci kümenin eleman sayısı kullanıcıdan istenir, sonra for döngüsüyle elemanlar alınır. İkinci küme için de aynısı yapılır. Asıl soru karşılaştırmanın nasıl yapılacağıdır. Hoca bunu "kağıt üzerinde elle çözseniz nasıl çözerdiniz?" diye sordu.

Cevap şudur: bir kümeden bir eleman alınır ve diğer kümede var mı diye bakılır. O eleman bitince aynı kümenin bir sonraki elemanı alınır ve yine diğer kümede aranır. Böylece bir kümenin bütün elemanları tek tek diğer kümede aranmış olur.

Hoca, buradaki iki dizinin de sırasız olduğunu ayrıca belirtti. Sıralı diziler için biraz daha farklı çözümler olacağını söyledi.

2. İç İçe İki For ile Akış Diyagramı

Hoca akış diyagramını öğrencilerle birlikte adım adım çizdi. Kullanıcıya verilen mesajları diyagrama yazmadı. Akış diyagramında diziler 1'den başlar. İlk çözüm şöyledir:

1. Başla. N ve M'yi oku.
2. I, 1'den N'e kadar: `kume1[I]` oku.
3. I, 1'den M'e kadar: `kume2[I]` oku.
4. `K = 1` ata.
5. I, 1'den N'e kadar (dış döngü):
   - J, 1'den M'e kadar (iç döngü):
     - `kume1[I] = kume2[J]` ise: `kesisim[K] = kume1[I]` ve `K = K + 1`.
     - Değilse bir şey yapılmaz.
6. I, 1'den K−1'e kadar: `kesisim[I]` yaz.
7. Dur.

`K` yeni bir dizinin, yani kesişim dizisinin indisidir. Bir eleman eklendikten sonra K bir artırılır ki bir sonraki turda dizinin bir sonraki gözünü göstersin. K'nın başlangıç değeri, döngülere girmeden önce atanmalıdır.

Bu çözüm üzerine sınıftan birkaç soru geldi.

Soru: If içinde neden çift eşittir kullanıldı? Normalde tek eşittir kullanılmıyor mu?

Cevap: Çift eşittir kullanılabilir, yanlış değildir. Ama o, C sözdizimine özel bir kullanımdır. Akış diyagramında genel kullanım tercih edilir, bu yüzden tek eşittir olarak düzeltildi.

Soru: Küme 2'nin elemanlarını okuduğumuz for'un içinde eşitliği de kontrol edip bir for'dan tasarruf etsek yanlış olur mu?

Cevap: Yanlış olmaz, gerçek hayatta böyle yazmak hata değildir. Bu derste okuma işiyle asıl iş birbirinden ayrı gösterilir, çünkü bu öğretici açıdan daha uygundur.

Soru: Küme 2'yi okurken de I kullanıldı. J kullanmamız gerekmiyor mu?

Cevap: Hayır. I birinci döngüde 1'den N'e gider, ikinci döngüde başlangıcı tekrar 1'e çekilir ve bu kez M'e gider. İki ayrı indis kullanmak zorunlu değildir. Kullanılırsa da yanlış olmaz.

Soru: Yazdırma döngüsü K'ya kadar değil, K−1'e kadar gitmemeli mi?

Cevap: Evet. Son eklemeden sonra K bir kez daha artırılır, yani döngüden çıkıldığında K, eleman sayısından bir fazladır. Yazdırma K−1'e kadar gitmelidir. Hoca bunu, sınır değerlerinin önemini gösteren güzel bir tespit olarak değerlendirdi.

3. İç Döngüyü While'a Çevirmek

Bir öğrenci iç döngüde for yerine while kullanmanın daha mantıklı olabileceğini söyledi. Başka bir öğrenci sebebini açıkladı: dış döngüde bütün elemanlara bakmak gerekir, ama iç döngüde eşit eleman bulunduğu anda iş bitmiştir.

Hoca bu açıklamayı kümenin tanımıyla bağladı. Küme denince genelde aynı elemanın tekrar etmediği kabul edilir. Veri seti olsaydı içinde tekrar olabilirdi; örneğin B, 6, 1, 7, 1 olabilirdi. Ama her iki durumda da bir eleman bulunduğunda onun ortak olduğu öğrenilmiştir. Geri kalanda ondan kaç tane olduğu bu soruyu ilgilendirmez. Bu yüzden iç for, while'a dönüştürülür.

3.1 While Koşulunu Kurmak

While'ın koşulu öğrencilerle birlikte kuruldu. İki parçası vardır:

- `J <= M`: küme 2'nin sınırları içinde kalmalıyız. Bir öğrenci önce `J < M` önerdi, başka bir öğrenci eşittirin eklenmesi gerektiğini söyledi. Akış diyagramında diziler 1'den M'e kadar kullanıldığı için doğrusu `J <= M`'dir.
- `kume1[I] ≠ kume2[J]`: farklı olduğu sürece aramaya devam edilir.

J'ye döngüden önce değer verilmelidir: `J = 1`.

Döngünün içine ne yazılacağı konusunda öğrencilerden farklı cevaplar geldi. "Hiçbir şey olmasına gerek yok" cevabı doğru değildir. Doğrusu `J = J + 1` yazmaktır. For kendi kendine artırır, while ise artırmaz. J artırılmazsa döngüde ilerlenemez.

> **Ek bilgi (derste anlatılmadı):** Koşulun önce `J <= M`, sonra karşılaştırma şeklinde yazılması büyük olasılıkla önemlidir. C'de && işlecinin sol tarafı yanlışsa sağ tarafı hiç değerlendirilmez. Bu sayede J dizinin sonunu geçtiğinde sınır dışındaki göz okunmaz. Koşullar ters sırada yazılırsa bu koruma ortadan kalkar.

3.2 While'dan Neden Çıktık?

Döngüden çıkınca bir öğrenci hemen `kesisim[K] = kume1[I]` yazmayı önerdi. Hoca bunun erken olduğunu gösterdi: döngüde iki kontrol vardır ve ikisi de sağlandığı sürece döngüde kalınır. Biri bozulunca çıkılır. Çıkış anında elemanın bulunup bulunmadığı bilinmez.

> **Hocanın vurgusu:** While'dan çıktıktan sonra neden çıkıldığını bilmiyoruz. Önce bunun kontrolü yapılmalıdır.

Bunu anlamanın yolu `J <= M` sorusunu (ya da tersi olan `J > M` sorusunu) sormaktır. `J <= M` doğruysa çıkış sebebi elemanların eşit olmasıdır, yani eleman bulunmuştur. True kolunda atama yapılır: `kesisim[K] = kume1[I]` ve `K = K + 1`. False kolunda yapılacak iş yoktur.

Düzeltilmiş dış döngü şöyledir:

1. I, 1'den N'e kadar:
   - `J = 1`
   - `J <= M` ve `kume1[I] ≠ kume2[J]` olduğu sürece: `J = J + 1`
   - `J <= M` ise: `kesisim[K] = kume1[I]` ve `K = K + 1`

Son elemanda da eşleşme yoksa J bir kez daha artar ve M'yi geçer. Bu yüzden koşul false olur ve false koluna gidilir. Bir öğrenci bunu özellikle sordu, hoca da aynı şekilde doğruladı.

Soru: `J <= M` koşulunu koymaya gerek var mı? Altına doğrudan atamayı yazsak olmaz mı?

Cevap: Soruyu soran öğrenci kendisi cevapladı: koşul olmazsa eleman bulunmadığında da ekleme yapılır. Hoca, bu tür kararları bir kez daha gözden geçirip kendi kendine tespit edebilmenin istenen şey olduğunu söyledi.

Soru: K'ya başta 0 mı, 1 mi verdik?

Cevap: Akış diyagramında 1 verildi, okuma döngülerinin hemen arkasında.

3.3 Çözümün Örnek Üzerinde İzlenmesi

$A = \{1, 3, 5, 2, 4\}$, $B = \{6, 1, 7, 2\}$ için, yani $N = 5$, $M = 4$ iken while'lı çözüm şöyle ilerler (ara adım eklendi):

I	`kume1[I]`	While'dan çıkışta J	`J <= 4`?	İşlem	K (sonra)
1	1	2	Evet	`kesisim[1] = 1`	2
2	3	5	Hayır	Yok	2
3	5	5	Hayır	Yok	2
4	2	4	Evet	`kesisim[2] = 2`	3
5	4	5	Hayır	Yok	3

Sonuçta K = 3 olur. Yazdırma 1'den K−1 = 2'ye kadar gider ve ekrana 1 ile 2 yazılır.

3.4 Alternatif: Flag Değişkeni

Bir öğrenci önceki algoritmalardaki yöntemi hatırlattı: karşılaştırma bir değişken üzerinden yapılabilir. Buna `flag` adı verilebilir. Önce 0 yapılır, içeride bir if ile küme 1 ve küme 2 karşılaştırılır, eşitlik bulununca flag set edilir. Döngüden çıkınca flag'e bakılır.

Hoca bunun da doğru olduğunu söyledi. İki bakış açısı da öğrenilsin diye kontrolü doğrudan while içine yazan çözümü göstermişti. İsteyen ikisinden birini kullanabilir.

4. Hangi Küme Dışta, Hangisi İçte Olmalı?

Bir öğrenci şunu sordu: bir kümede birkaç milyon, diğerinde beş on eleman varsa bu önemli mi, döngüleri buna göre ayarlamak gerekir mi? Hoca soruyu sınıfa açtı. 5 ve 10 bin elemanlı iki küme için hangisi içte olmalıdır, yoksa fark etmez mi?

İlk akla gelen sezgi, küçük kümeyi büyük kümenin içinde aramanın daha mantıklı olduğudur: küçük kümenin elemanları büyük kümenin başındaysa onlar çok daha erken bulunur. Hoca, böyle bir izlenim oluşsa da bunu masaya yatırmak gerektiğini söyledi ve bir örnek verdi. Küçük küme 1, 3, 5, 7 olsun. Büyük küme 10, 100, 102, 105 diye devam etsin, 1, 3, 5, 7 de sonunda olsun.

Hoca iki sırayı şöyle saydı:

- **Büyük küme dışta:** 10 alınır; 1, 3, 5, 7 ile karşılaştırılır (4 karşılaştırma). 100 ve 102 de aynı şekilde alınır (toplam 12). 105 de alınınca toplam 16 olur. Arkasından 1, 3, 5, 7 gelir ve tek tek bulunur.
- **Küçük küme dışta:** 1, 10, 100, 102, 105 ile karşılaştırılır (4). 3, 5 ve 7 için de 4'er karşılaştırma yapılır. Toplam $4 \times 4 = 16$ olur.

Hoca buradan, iki taraftan da ikili karşılaştırma yapıldığı ve eşleşme görülen yerde çıkıldığı için hangi kümeden başlandığının önemli olmadığı sonucuna vardı. While kullanıldığında da bu sonucun değişmediğini söyledi.

> **Ders sonrası not:** Hoca küçük küme dıştayken 16 saydı; 1, 3, 5, 7 büyük kümenin sonunda olduğu için bu sayı büyük olasılıkla yalnız ilk dört elemanla yapılan karşılaştırmaları içerir. While ile tam sayım şöyledir (python ile doğrulandı). Büyük küme dıştayken $16+1+2+3+4 = 26$, küçük küme dıştayken $5+6+7+8 = 26$ karşılaştırma yapılır. Yani bu örnekte hocanın sonucu doğrudur, iki sıra eşittir, ama toplam 16 değil 26'dır. İç for ile her iki sırada da $8 \times 4 = 32$ karşılaştırma yapılır. Eşleşmelerin yerine göre iki sıranın sayısı büyük olasılıkla her girdide tam eşit çıkmaz; "sıra hiç fark etmez" sonucu bu örnek için geçerlidir.

Hoca kesişim sorusunu şöyle özetledi: iki for ile while arasındaki fark görüldü, hangi dizinin önce alınacağı konuşuldu ve sınır değerlerinin ekrana yazdırırken önemli olduğu vurgulandı. Ardından bu çözümün kodunun yazılması için ara verildi.

5. Akış Diyagramından C Koduna

Aradan sonra hoca, bir öğrencinin yazıp gönderdiği kodu ekranda birlikte inceledi. Önce genel bir ilke söyledi.

> **Hocanın vurgusu:** Bir algoritmayı tasarlayıp akış diyagramını çizdiğinizde en kolay şey onun kodunu yazmaktır. Yapmanız gereken, C sözdizimine uygun komutları yerli yerine oturtmaktır. Akış diyagramı ya da algoritma olmasaydı zorlanabilirdiniz.

Kod yazarken bir IDE ya da çevrimiçi bir araç kullanılması, deftere yazmakla yetinilmemesi de tavsiye edildi.

5.1 Tanımlama Kısmı ve Tipler

Akış diyagramında tanımlanmayan değişkenlerin hepsi programlama dilinde tanımlanmalıdır. Kullanılabilecek tipler şunlardır:

- `int`: tam sayı.
- `float`: kesirli sayı.
- `double`: daha hassas kesirli sayı.
- `char`: karakter.

Soru: Tanımlamaları kodun aralarında yapabilir miyiz?

Cevap: Yapılabilir, ama kodlama düzeni için uygun değildir. Tanımlama ve ilklendirme genelde ana programın başında yazılır. Başka kişilerle ortak çalışıldığında aralara serpiştirilmiş tanımlar büyük sıkıntı yaratır ve fark edilmesi zordur.

5.2 Dizinin Boyutu Neden Sabit Verilir?

Dizi tanımlanırken köşeli parantez kullanılır ve içine mutlaka sabit bir sayı yazılır. Örneğin `int Kume1[50];`. Parantezin içine N ya da M gibi bir değişken adı yazılmamalıdır. Hoca, "ben yazdım, çalışıyor" denebileceğini ve bazı platformlarda çalışıyor olabileceğini, ama doğrusunun bu olmadığını söyledi.

Yapılması gereken, bir sayı seçip büyükçe vermektir. Şu an 500 elemanı deneme şansı olmadığı için 50 yeterlidir. Kullanıcının bu değerden daha küçük bir değer gireceği varsayılır.

Aynı konuda iki kez soru geldi.

Soru: Kullanıcıdan N'in değerini aldıktan sonra diziyi `Kume1[N]` diye tanımlayamaz mıyız? Eleman sayısını bilmeden nasıl tanımlıyoruz?

Cevap: Bu haliyle tanımlanamaz. Dizi tanımının kullanıcıdan alınan bilgiyle bir ilgisi yoktur: kullanıcıdan bağımsız, büyükçe bir değer belirlenir. Sabit boyutlu tanımı değerleri aldıktan sonraya taşımak yanlış değildir, ama düzen açısından böyle olmamalıdır. Parantezin içine N yazmak ise yanlıştır.

Soru: Sonradan tanımlamadaki problem tam olarak nedir?

Cevap: Bu tür tanımda bellekte yer ayırma işi daha program başlamadan gerçekleşir. Kullanıcıdan değer alındığı anda bir yer ayırma yapılmaz. O sırada N ve M'nin değeri de bilinmediği için sistem yer ayıramaz. İleride görülecek dinamik bellek ayırma (dynamic memory allocation) yönteminde yer çalışma sırasında ayrıldığı için bu sorun ortadan kalkacaktır.

> **Ek bilgi (derste anlatılmadı):** Hocanın "bazı platformlarda çalışır" dediği durum büyük olasılıkla C99 ile gelen değişken uzunluklu dizilerdir (variable-length array). gcc gibi derleyicilerde N okunduktan sonra yazılan `int Kume1[N];` çalışma sırasında yer ayırır. Bu özellik her derleyicide bulunmaz, bu yüzden dersin kuralı güvenli olandır. N okunmadan önce yazılan `int Kume1[N];` ise değeri belli olmayan bir boyut kullanır ve kesinlikle hatalıdır.

5.3 Printf ve Scanf

`printf` ile çift tırnak içinde ekrana ne yazılacağı belirtilir. Dışarıdan değer okurken, okunacak değişkenin adı yazılır ve başına `&` (ampersand) konur. Ampersand değişkenin adresini ifade eder. Okuma, değişkenin adresine yapıldığı için okumalarda bu işaret kullanılır.

Döngü içindeki mesajda `%d` yerine, ondan sonra yazılan ifadenin değeri gelir. Burada `i + 1` yazılmıştır, çünkü kullanıcıya birinci, ikinci diye göstermek istenir ve `i` 0'dan başlar. Arka arkaya birden fazla biçim belirteci de yazılabilir: iki `%d` ya da bir `%d` ve bir `%f` olabilir. İlk belirtece birinci ifade, ikincisine ikinci ifade karşılık gelir.

5.4 Sınır Değerleri: 1'den Başlamak ve 0'dan Başlamak

Öğrencinin kodunda okuma döngüsü `i <= N-1` diye yazılmıştı. Bu yanlış değildir. Daha yaygın kullanım `i < N`'dir.

Hoca `k` başlangıcında önce kendisi yanılıp 1 olması gerektiğini söyledi, sonra hemen düzeltti: akış diyagramında 1 denmişti ama burada dizinin ilk elemanı sıfırıncı göze yerleşir. Bu yüzden `k = 0` doğrudur ve öğrenci doğru yazmıştır.

Soru: Az önce `j <=` mi yazdık? Burada sadece `j <` yazıyor.

Cevap: Evet, `j < M` yazıyor. Küçük eşit yazmak istenseydi `j <= M-1` yazılması gerekirdi, çünkü dizilerde 0 ile $M-1$ aralığı kullanılır. Akış diyagramında ise 1 ile M aralığı kullanılmıştı.

> **Hocanın vurgusu:** Buradaki sınır değerleri akış diyagramından farklıdır. Akış diyagramında diziler 1'den başlar, C'de 0'dan başlar. Bu yüzden C'de hep eksi bire kadar gidildiğini varsayın.

Çevirinin özeti şöyledir:

Akış diyagramı	C kodu
`J = 1`	`j = 0`
`J <= M`	`j < M` ya da `j <= M-1`
`K = 1`	`k = 0`
Yazdırma: 1'den K−1'e	Yazdırma: 0'dan k−1'e (`i <= k-1` ya da `i < k`)

5.5 Yazdırma Döngüsündeki Sınır Hatası

Ekrandaki kodda yazdırma döngüsünün sınırında bir hata vardı. Bir öğrenci kendi denemesinde bir elemanın yazılmadığını söyledi ve küçük eşit `k-1` olması gerektiğini önerdi.

Hoca uç değerle kontrol etti: `k` 0'dan başlar ve her eklemede bir artar. Tek eleman yazdırılacaksa `k` 1 ile biter. Bu durumda `k-1` 0'dır; `i < 0` koşulu hiç sağlanmaz ve eleman yazılmaz (ara adım eklendi). `i <= k-1` ise `i = 0` için bir kez çalışır. Hoca öğrencinin haklı olduğunu ve hatanın kendisine ait olduğunu söyledi. A ve B örneğinde hatalı sınır yalnız 1'i, doğru sınır 1 ve 2'yi yazar (ara adım eklendi).

> **Hocanın vurgusu:** Uç değerleri mutlaka kontrol edin.

5.6 Koşullarda Parantez, Girinti ve Boş Else

While koşulundaki her koşul ayrıca parantez içine alınmalıdır: `while ((j < M) && (Kume1[i] != Kume2[j]))`. Bu, sözdizimi açısından zorunlu değildir. Ama işleçlerin öncelikleri birbirinden farklı olabilir ve bunları bilmeyebilirsiniz. Mantığı garanti altına almak için parantez kullanılmalıdır.

Soru: While ve if satırlarında bir girinti (indentation) olması gerekmiyor muydu?

Cevap: Bu if, while ile aynı hizadadır. For'un içinde hem while hem if vardır. For'a ait bütün işlemler aynı hizada yazılır. If, while'ın içinde olsaydı o zaman girinti yapmak uygun olurdu.

Soru: If'in false koluna ne yazdık, nasıl belirteceğiz?

Cevap: Akış diyagramında false koluna bir şey yazılmamıştı. Yazılmadığında zaten bir şey yapılmaz. False kolunda iş olsaydı `else` denip parantez açılır ve işler oraya yazılırdı. İş yoksa boşuna `else` açılmaz.

Soru: Çıktıda sayılar yan yana, tek bir sayı gibi çıkıyor. Ayıracak mıyız?

Cevap: Bunun biçimli yazımı vardır; `%2d` gibi yazılabilir. Hoca bunun yerine `printf`'e `\n` ekledi. Her seferinde bir satır atlanır ve sayılar yan yana çıkmaz. Dersin amacı C'nin bütün yeteneklerini aktarmak değil, çizilen akış diyagramlarını koda dökebilecek kadar C'yi göstermektir.

5.7 Kesişim Kümesinin C Kodu

Aşağıda derste incelenen kodun düzeltilmiş hâli vardır: yazdırma sınırı `k-1`, while koşulları parantezli ve çıktıda `\n` kullanılmıştır. Kod `gcc -Wall` ile derlendi. A ve B örneğinde 1 ve 2 çıktısını verdi. Hoca kodu derste çalıştırmadı.

```c
#include <stdio.h>

int main()
{
    int N, M;
    int Kume1[50], Kume2[50];
    int kesisim[50];
    int i, j;
    int k = 0;

    printf("Ilk kumenin eleman sayisini girin: ");
    scanf("%d", &N);
```

```
    printf("Ikinci kumenin eleman sayisini girin: ");
    scanf("%d", &M);

    for (i = 0; i < N; i++)
    {
        printf("1. kumenin %d. degerini girin: ", i + 1);
        scanf("%d", &Kume1[i]);
    }
    for (i = 0; i < M; i++)
    {
        printf("2. kumenin %d. degerini girin: ", i + 1);
        scanf("%d", &Kume2[i]);
    }

    for (i = 0; i < N; i++)
    {
        j = 0;
        while ((j < M) && (Kume1[i] != Kume2[j]))
        {
            j = j + 1;
        }
        if (j < M)
        {
            kesisim[k] = Kume1[i];
            k = k + 1;
        }
    }

    for (i = 0; i <= k - 1; i++)
    {
        printf("%d\n", kesisim[i]);
    }
    return 0;
}
```

Kod paylaşılırken hoca, kaynak dosyanın gönderilmesi gerektiğini de söyledi. Çalıştırılabilir dosya gönderilirse diğerlerinde çalışmaz.

6. Çok Boyutlu Diziler ve Matrisler

Şimdiye kadar diziler tek boyutlu olarak görülmüştü. Sırada iki, üç, dört boyutlu diziler (multi-dimensional arrays) vardır. İki boyutlu dizi (2-D array), karşımıza matris olarak çıkar. Satırlar ve kolonlar (sütunlar) vardır.

Satır sayısı N, sütun sayısı M ise ilk göz `mtr[0][0]`, son göz `mtr[N-1][M-1]` olarak adlandırılır.

Tanım yine köşeli parantezle yapılır. İlk köşeli parantez satır sayısını, ikincisi sütun sayısını verir. `int mtr[2][3];` tanımı 2 satır ve 3 sütundan oluşan bir matris verir. Toplam $2 \times 3 = 6$ eleman vardır:

	Sütun 0	Sütun 1	Sütun 2
Satır 0	`mtr[0][0]`	`mtr[0][1]`	`mtr[0][2]`
Satır 1	`mtr[1][0]`	`mtr[1][1]`	`mtr[1][2]`

6.1 Matris Bellekte Nasıl Saklanır?

Kağıda çizdiğimiz iki boyutlu görünüm soyut bir gösterimdir. Matris bellekte bu şekilde saklanmaz. Önce birinci satırın elemanları sütun sırasıyla, sonra ikinci satırın elemanları sütun sırasıyla arka arkaya yerleşir:

`mtr[0][0], mtr[0][1], mtr[0][2], mtr[1][0], mtr[1][1], mtr[1][2]`

Hocanın örneğiyle, gözlerdeki değerler sırasıyla 5, 3, 2, 1, 0, 9 ise (birinci satır 5, 3, 2; ikinci satır 1, 0, 9) bellekte de 5, 3, 2, 1, 0, 9 olarak yer alır.

Soru: Saklanma şeklini tekrar söyleyebilir misiniz?

Cevap: Belleğe yerleşme birinci satırın elemanlarından başlar. Birinci satırın ilk sütunu, ikinci sütunu ve bu şekilde son sütunu yerleşir. Sonra ikinci satır gelir. Satırlar sütunları sırayla takip ederek arka arkaya yerleşir.

6.2 Daha Fazla Boyut

Köşeli parantez eklemek yeni bir boyut yaratmak demektir. Örnek: `int kup[4][5][7];`. Eleman sayısı $4 \times 5 \times 7 = 140$'tır.

Bu yapıyı bir küp gibi, x, y, z eksenleriyle düşünebilirsiniz. Fiziksel olarak üç boyutlu bir ortamda yaşadığımız için, fiziksel ortamı modellemekte üç boyutlu diziler daha sık karşımıza çıkar. Dört, beş, altı boyutun kullanılabileceği yerler vardır, ama bu kullanımlar çok sık görülmez.

7. Örnek Soru: Matrisin Kolon Toplamları

Soru şudur: verilen bir matrisin kolon değerlerini toplayıp yeni bir dizide saklayan algoritmanın akış diyagramını çiziniz.

> **Hocanın vurgusu:** Bu tür sorularda önce bir matris çizin. Her türlü soru için, önce kağıt üstünde bu işi nasıl yapacağınızı düşünün.

Birinci kolon, ikinci kolon, üçüncü kolon ve dördüncü kolon ayrı ayrı toplanır, toplamlar da bir diziye aktarılır. Bir öğrenci iki for döngüsü gerektiğini söyledi. Hoca neye dikkat edileceğini sordu: içte ne dönecek, dışta ne dönecek? Cevap: **içte satırlar, dışta sütunlar** döner.

7.1 Akış Diyagramının Kurulması

Önce N (satır sayısı) ve M (sütun sayısı) okunur. Sonra matris okunur. Okumada dış döngü satırlarda, iç döngü sütunlardadır. Toplama için dış döngü M'e, iç döngü N'e kadar gider. Her sütun için bir toplam değişkeni gerekir ve başlangıç değeri 0'dır.

İlk denemede toplama satırı `toplam = toplam + matris[I][J]` diye önerildi. Hoca bunu sınıfla örnek üzerinde test etti. Tahtadaki matris sütun sütun şöyle yazıldı: 5, 7, 8, 9; 3, 2, 1, 0; 6, 4, 7, 3; 8, 2, 1, 0.

$$\begin{bmatrix} 5 & 3 & 6 & 8 \\ 7 & 2 & 4 & 2 \\ 8 & 1 & 7 & 1 \\ 9 & 0 & 3 & 0 \end{bmatrix}$$

I = 1, J = 1 için `[1][1]`'e erişilir. Döngü dönünce J = 2 olur ve `[1][2]`'ye erişilir. Bu, birinci sütunda aşağı inmek yerine birinci satırda sağa gitmek demektir. Demek ki burada bir sorun vardır.

Bir öğrenci for döngülerinde I ile J'nin yerini değiştirmeyi önerdi. Başka bir öğrenci ise döngülere dokunmadan indislerin yerini değiştirmeyi önerdi: `matris[J][I]`. Hoca ikinciyi denedi. `[1][1]` ile başlanır, 5 alınır. Sonra J değişir ve `[2][1]`, `[3][1]`, `[4][1]` alınır. Birinci sütun böylece toplanmış olur.

7.2 Toplam Nereye Yazılır?

İç döngü bittikten sonra yapılacak iş, toplamı diziye aktarmaktır. Bir öğrenci diziye ayrı bir döngüde, dışarıda da yazılabileceğini önerdi. Bu yapılamaz: bir tur bitince yalnız o sütunun toplamına sahibizdir. İkinci tura geçince bu değer kaybolur. Bu yüzden değer iç döngüden çıkar çıkmaz saklanmalıdır: `dizi[I] = toplam`.

Ayrı bir K sayacına gerek yoktur, çünkü dış döngünün I değeri de birer birer artar. Son olarak bir döngüyle dizi yazdırılır. Tam akış şöyledir:

1. Başla. N ve M'yi oku.
2. I, 1'den N'e kadar; J, 1'den M'e kadar: `matris[I][J]` oku.
3. I, 1'den M'e kadar (sütunlar):
   - `toplam = 0`
   - J, 1'den N'e kadar (satırlar): `toplam = toplam + matris[J][I]`
   - `dizi[I] = toplam`
4. I, 1'den M'e kadar: `dizi[I]` yaz.
5. Dur.

Tahtadaki matris için sonuç şöyle hesaplanır (ara adım eklendi):

- 1. sütun: $5 + 7 + 8 + 9 = 29$
- 2. sütun: $3 + 2 + 1 + 0 = 6$
- 3. sütun: $6 + 4 + 7 + 3 = 20$
- 4. sütun: $8 + 2 + 1 + 0 = 11$

Dizi 29, 6, 20, 11 olur.

7.3 Soru ve Cevaplar

Soru: En dış döngüde I yerine J kullanmak iyi bir pratik mi?

Cevap: I ve J'nin yerleri sabit olmak zorunda değildir. Dışa J, içe I yazılabilir. Önemli olan içeride satırların, dışarıda sütunların dönmesi ve çözümün doğru çalışmasıdır.

Soru: `toplam` kullanmadan `dizi[I] = dizi[I] + matris[J][I]` yazabilir miyiz?

Cevap: Evet, böylece `toplam` değişkeninden kurtulunur. Ancak ikinci bir önemli nokta vardır: dizinin ilk değeri yoktur. Bu yüzden iç içe döngülerden önce ayrı bir döngüyle dizinin 1'den M'e kadar bütün gözleri 0'a atanmalıdır.

> **Hocanın vurgusu:** Bu yazımı kullanıyorsanız dizinin gözlerini mutlaka önceden sıfırla ilklendirmeniz gerekir. Bu önemlidir.

Soru: Matrisi okurken N dışta, M içteydi. Toplarken neden tersi oldu?

Cevap: Okuma satır satır yapılır, bu yüzden önce satır gezilir. Toplamada ise bir sütunda aşağı inilmek istenir. Bu yüzden `matris[I][J]` yerine `matris[J][I]` yazıldı. İç döngüde satırlar değişir, dış döngüde bir sütun kayılır.

Soru: İkinci iç içe döngüde de önce N'e, sonra M'e kadar gitseydik olur muydu?

Cevap: "Olur muydu" sorusunun cevabı önce analiz yaparak bulunur. İçeride satırlar üzerinde ilerlemek gerekir, bu yüzden içeride satır sayısı olan N, dışarıda sütun sayısı olan M vardır. Yerleri değiştirilirse doğru çalışmaz.

8. Örnek Soru: Matris Simetrik mi?

Soru şudur: verilen bir matrisin simetrik olup olmadığını bulan algoritmanın akış diyagramını çiziniz.

Simetrik matriste köşegen bir ayna gibi düşünülür. Köşegenin bir tarafındaki değerler, diğer taraftaki ayna konumundaki değerlerle aynı olmalıdır. Bunlardan biri bile farklıysa matris simetrik değildir. Matris $N \times N$, yani kare olmalıdır. Bu yüzden dışarıdan yalnızca tek bir boyut okunur.

8.1 For mu, While mı?

İç içe iki döngü gerektiği açıktır. Soru, bunların for mu while mı olacağıdır. Bir öğrenci while önerdi, başka bir öğrenci de gerekçesini söyledi: simetrik olmayan bir yer bulunduğunda hemen çıkılıp "simetrik değil" denebilir. Hoca, ekrandaki simetrik olmayan örnekte ilk kontrol yapıldığı anda kararın verildiğini, geri kalan kontrollere ihtiyaç kalmadığını gösterdi.

Hocanın vurgusu: Adım sayısı belli olmadığı için burada while kullanılması gerekir.

Çözümü ilk öneren öğrenci, `I < N` ve eşitlik koşuluyla dış while, `J <= N` ve eşitlik koşuluyla iç while önermişti; iç döngüde J, `I + 1`'den başlıyordu. Hoca bu çözümün mantıklı göründüğünü söyledi, ama `flag` değişkeniyle biraz farklı bir yapı kullandı.

> **Ders sonrası not:** Bu önerinin dış while koşulunda eşitlik de kontrol ediliyorsa, ilk girişte J'nin henüz değeri yoktur ve sonraki girişlerde J dizinin sınırını geçmiş olabilir. Büyük olasılıkla hocanın flag'li yapısı bu açıdan daha güvenlidir. Hoca öneriye sonradan geri dönmedi.

8.2 Hangi Gözlere Bakılmalı?

İlk akla gelen çözüm matrisi satır satır taramaktır. Bu çözüm yanlış değildir, ama dezavantajları vardır:

- Ek bir kontrol konulmazsa köşegen de taranır. Bir eleman kendisiyle karşılaştırılır.
- Her çift iki kez kontrol edilir. Önce `m[I][J]` ile `m[J][I]` karşılaştırılır. Sonra o satıra gelindiğinde aynı çift tersten bir daha karşılaştırılır.

Bu yüzden yalnızca köşegenin üst kısmına bakmak yeterlidir, çünkü bu gözlerin her birinin karşılığı bu kontrolle zaten görülmüş olur. I. satırdaysak sütun `I + 1`'den başlar ve sona kadar gider. Bir öğrenci bunu `I` ile `J` arasında bir kural olarak koymayı önerdi; hoca bunu `J = I + 1` başlangıcıyla kurdu.

Son satıra bakmaya gerek yoktur. Son satırda köşegenden sonra başka göz kalmaz, eşleşecek bir yer olmaz. Bu yüzden dış döngünün koşulu `I <= N` yerine `I < N` olur.

8.3 İçeride Ne Yapılır?

Bir öğrenci `m[I][J] = m[J][I]` kontrolünün true kolunda "simetriktir" yazmayı önerdi. Bu yanlıştır. Bir çiftin eşit olması henüz bir şey söylemez, çünkü bütün çiftlere bakılması gerekir. Kontrol tersine çevrilir: `m[I][J] ≠ m[J][I]` ise true kolunda `flag = 0` yapılır. Else kolunda yapılacak iş yoktur.

Hoca çizimi bitirdikten sonra bir öğrenci J ve I'nın artırılması gerektiğini fark etti. İki artırma da akışa eklendi. Son hâli şöyledir:

1. Başla. N'i oku.
2. I, 1'den N'e kadar; J, 1'den N'e kadar: `m[I][J]` oku.
3. `flag = 1`, `I = 1`.
4. `I < N` ve `flag = 1` olduğu sürece:
   - `J = I + 1`
   - `J <= N` ve `flag = 1` olduğu sürece:
     - `m[I][J] ≠ m[J][I]` ise `flag = 0`.
     - `J = J + 1`
   - `I = I + 1`
5. `flag = 1` ise "Simetrik" yaz, değilse "Simetrik değil" yaz.
6. Dur.

8.4 Soru ve Cevaplar

Soru: İç while'da `flag = 1` yerine doğrudan eşitlik koşulunu yazıp, while'dan çıkınca `J <= N` kontrolü yapsak olur mu?

Cevap: Olur gibi görünüyor. Bu durumda eşit olduğu ve `J <= N` olduğu sürece dönülür, hangi sebeple çıkıldığı sonradan kontrol edilir. Hoca bunu kafadan söylediğini, garanti için analiz yapmak gerektiğini ekledi.

Soru: 4×4 ya da 5×5 kare matriste de çalışır mı?

Cevap: Çalışır. İsteyen deneyebilir.

Soru: Matrisi okurken J, N'e kadar gitti. I'yı 1'e eşitledik ama J hâlâ son değerinde kalmıyor mu?

Cevap: J, iç döngüye girmeden hemen önce `J = I + 1` ile değiştirilir. Bu yüzden bir sorun oluşmaz.

Hoca bu iki sorunun kodunu öğrencilere bıraktı. Matrisin nasıl tanımlandığı ve gözlerine nasıl erişildiği gösterilmişti.

> **Ek bilgi (derste anlatılmadı):** Derste yazılmayan iki kodun büyük olasılıkla en doğrudan C karşılığı aşağıdadır. Sınırlar, 5. bölümde anlatılan çeviriyle 0'dan başlatılmıştır. İkisi de `gcc -Wall` ile derlendi. Kolon programı tahtadaki matris için 29, 6, 20, 11 verdi.

```c
/* Kolon toplamlari */
#include <stdio.h>

int main()
{
    int N, M, i, j, toplam;
    int matris[50][50], dizi[50];

    scanf("%d %d", &N, &M);
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++)
            scanf("%d", &matris[i][j]);

    for (i = 0; i < M; i++)        /* dis dongu: sutunlar */
    {
        toplam = 0;
        for (j = 0; j < N; j++)    /* ic dongu: satirlar */
            toplam = toplam + matris[j][i];
        dizi[i] = toplam;
    }

    for (i = 0; i < M; i++)
        printf("%d\n", dizi[i]);
    return 0;
}
```

```c
/* Simetri kontrolu */
#include <stdio.h>

int main()
{
    int N, i, j, flag;
    int m[50][50];

    scanf("%d", &N);
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            scanf("%d", &m[i][j]);

    flag = 1;
    i = 0;
    while ((i < N - 1) && (flag == 1))
    {
        j = i + 1;
        while ((j < N) && (flag == 1))
        {
            if (m[i][j] != m[j][i])
                flag = 0;
            j = j + 1;
        }
        i = i + 1;
```

```
        }

        if (flag == 1)
            printf("Simetrik\n");
        else
            printf("Simetrik degil\n");
        return 0;
    }
```

9. Scanf ve Printf Üzerine Son Sorular

Soru: `scanf`'te `%d`'den sonra bir boşluk bırakınca sorun çıkıyor. Nedeni nedir?

Cevap: Hoca, sorunun kaynağını kesin söyleyebilmek için kodu görmesi gerektiğini belirtti. Sohbette `%4d`, `%3d` gibi biçimli yazım önerildi. Hoca bunun boşluk bırakmayı ve hizalı yazmayı sağladığını, ama sorunun biraz farklı olduğunu düşündüğünü söyledi. Boşluğun `%d`'ye nasıl eklendiğinin görülmesi gerekiyordu.

> **Ek bilgi (derste anlatılmadı):** Burada büyük olasılıkla `scanf("%d ", &x);` gibi bir kullanım söz konusudur. `scanf` biçim dizgisindeki boşluğu, boşluk olmayan bir sonraki karakter gelene kadar beklemek olarak yorumlar. Bu yüzden program bir değer daha girilene kadar donmuş gibi görünür. Biçim dizgisinin sonuna boşluk koymamak bu durumu önler.

Soru: İki girdi tek `scanf` ile nasıl alınır?

Cevap: `scanf("%d %d", &sayi1, &sayi2);` şeklinde okunabilir. Sohbete yazılan örnekte ikinci değişkenin önünde `&` yerine yanlışlıkla `%` vardı. Hoca önce sözdizimi hatası görmedi, sonra bir öğrencinin uyarısıyla hatayı fark edip düzeltti. Her iki değişkenin önünde de ampersand olmalıdır. Hoca bunu gözünün o işareti otomatik olarak ampersand okumasına bağladı.

Dersin Toparlaması

Bu derste bir problemin çözümü üç aşamada izlendi: kağıt üzerinde düşünmek, akış diyagramını çizmek ve C koduna çevirmek. Kesişim kümesi sorusunda iç içe for'dan while'a geçilerek eşleşme bulunduğu anda aramanın nasıl kesildiği, while'dan neden çıkıldığının `J <= M` ya da `flag` ile nasıl anlaşıldığı görüldü. Koda geçerken 1'den başlayan akış diyagramı sınırlarının 0'dan başlayan C sınırlarına nasıl çevrildiği, uç değer kontrolünün neden önemli olduğu ve dizi boyutunun neden sabit verildiği konuşuldu.

İkinci kısımda iki boyutlu diziler, yani matrisler tanıtıldı ve satır satır saklandıkları gösterildi. Kolon toplamları sorusu, dış döngüde sütunlar, iç döngüde satırlar dönerek çözüldü. Simetri sorusu ise köşegenin üstüne bakmanın yettiğini ve `flag` ile erken çıkışı gösterdi. Hoca, bundan sonraki süreçte matrislerle ilgili başka sorular çözüleceğini ve artık program yazılabilecek noktaya gelindiğini söyledi. Dinamik bellek ayırma da ileride görülecektir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.